UADE



TP Integrador Obligatorio

INGENIERIA DE DATOS I

TURNO: NOCHE

GRUPO: Cafetear

Integrantes del Grupo:

- L 1247201 – Joaquín Morales Pessacq
- L _____ – Camila Pogonza
- L _____ – Guadalupe Olivera
- L _____ – Lautaro Patricio Rojas
- L _____ – Alexander Demmerle
- L _____ – Santiago Derito

Profesor:
Ing. Franco Emanuel Salazar

Fecha de Entrega:05/11/2026

Trabajo Practico
Integrador Obligatorio

Índice

1. Resumen Ejecutivo 3
2. Introducción 3
2.1. Contexto 3
2.2. Problemática 3
2.3. Solución propuesta 4
3. Objetivos del Sistema 4
4. Requisitos del Sistema 5
4.1. Requisitos Funcionales 5
4.2. Requisitos No Funcionales 5
5. Diseño de la Base de Datos 6
5.1. Modelo Conceptual 6
5.2. Modelo Lógico 6
5.3. Modelo Físico 7
5.4. Diccionario de Datos 8
5.5. Dependencias Funcionales 11
6. Implementación y Scripts SQL 12
6.1. Inserción de Datos 12
6.2. Consultas SQL 13
6.3. Triggers y Vistas 16
6.4. Funciones y Procedimientos 17
7. Interfaz de Usuario 19
8. Plan de Desarrollo y Metodología 20
9. Optimización y Rendimiento 20
10. Seguridad y Recuperación 21
10.1. Roles y permisos 21
10.2. Respaldo y recuperación 21
11. Pruebas y Validación 22
12. Gestión de Riesgos 23
13. Costos y Presupuesto 23
14. Conclusiones y Recomendaciones 23
15. Referencias y Anexos 24

Trabajo Practico
Integrador Obligatorio

1. Resumen Ejecutivo

Cafetear es una base de datos relacional en SQL Server que registra la calibración de la máquina de espresso de una cafetería de especialidad y la opinión de los clientes sobre cada café servido. Hoy ese conocimiento vive en la memoria de cada barista: cuando alguien se va, o cuando se abre una sucursal, se pierde lo que se había aprendido sobre cómo extraer cada lote.

El sistema guarda, para cada **calibración** (dosis, tiempo de extracción y rendimiento en taza), qué barista la hizo, con qué **lote** de café, y cuántas estrellas le puso cada **cliente**. Con eso se puede responder qué receta funciona mejor para cada grano, cómo evoluciona la calidad mes a mes, qué lotes decepcionan y qué baristas necesitan apoyo.

Resultados de esta entrega: 7 tablas con datos de prueba reproducibles (10 proveedores, 10 granos, 40 lotes, 10 baristas, 60 clientes, 225 calibraciones y 346 calificaciones), 35 procedimientos CRUD y 2 de reporte, 4 vistas, 4 funciones, 2 triggers, 7 índices, 3 roles de seguridad y una copia de respaldo verificada. Todo se levanta con un solo comando (`docker compose up`) y todos los integrantes obtienen exactamente los mismos datos.

2. Introducción

2.1.Contexto

Una cafetería de especialidad ajusta su molino y su máquina cada día: la dosis de café molido, el tiempo de extracción y los gramos que caen en la taza cambian con cada lote, con la frescura del tostado y hasta con el clima. A ese ajuste se lo llama **calibración**. El negocio se complica porque los baristas rotan con frecuencia y se abren y cierran locales, de modo que la receta que funcionaba rara vez se transmite.

2.2.Problemática

- No hay registro histórico de qué dosis, tiempo y rendimiento se usaron con cada lote.
- La opinión del cliente no se vincula con la forma en que se preparó el café, así que no se sabe qué calibraciones agradan más.
- No se puede comparar el desempeño entre baristas ni detectar a tiempo que un lote nuevo está saliendo mal.
- Cuando un barista se va, se pierde su conocimiento; el reemplazo empieza de cero.

2.3.Solución propuesta

Una base de datos que relaciona **Proveedor → Lote → Calibración → Calificación**, con baristas y clientes como actores. Está pensada para mejorar los resultados y la calidad que recibe el cliente: la pregunta central es qué extracciones funcionan mejor y cómo se comportan los clientes al calificarlas.

Decisión de diseño: se descartaron las mezclas (blends) de granos para no complicar el modelo; cada lote corresponde a un único grano. Se agregó la entidad **Lote** porque la misma variedad cambia entre bolsas según el tostado y la frescura, y eso afecta la receta.

3. Objetivos del Sistema

Objetivo general: centralizar el conocimiento de calibración y la opinión del cliente para sostener y mejorar la calidad de la taza.

- Registrar cada calibración con sus tres variables (dosis, tiempo, rendimiento) y asociarla a un barista y a un lote.
- Registrar las calificaciones (1 a 5) de los clientes y vincularlas a la calibración que las originó.
- Identificar la receta de referencia de cada grano y la evolución mensual de la calidad.
- Detectar lotes problemáticos, baristas que necesitan capacitación y clientes con criterios opuestos.
- Garantizar integridad con triggers y claves, y separar permisos por perfil.
- Permitir que cualquier integrante levante el entorno con los mismos datos en minutos.

4. Requisitos del Sistema

4.1. Requisitos Funcionales

ID	Requisito	Cómo se resuelve
RF-01	Alta, baja, modificación y consulta de proveedores, granos, lotes, baristas, clientes, calibraciones y calificaciones	35 procedimientos `usp_<Tabla>_Crear/Obtener/Listar/Actualizar/Eliminar`
RF-02	Calcular la relación de extracción (rendimiento ÷ dosis)	Función `fn_RelacionExtraccion`
RF-03	Conocer el puntaje promedio de cada lote y calibración	Función `fn_PuntajePromedioLote` y vista `vw_PuntajePorCalibracion`
RF-04	Obtener la mejor receta de cada grano	Vista `vw_MejorCalibracionPorGrano`
RF-05	Ranking de baristas por período y resumen de un lote	Procedimientos `usp_RankingBaristas` y `usp_ReporteLote`
RF-06	Rechazar datos inválidos (puntaje fuera de rango, calificación repetida, calibrar con un lote que no existe aún o agotado)	Triggers `trg_Calificacion_Control` y `trg_Calibracion_ControlLote`
RF-07	Ver lo calibrado en el día	Vista `vw_CalibracionesDeHoy`

4.2. Requisitos No Funcionales

ID	Requisito	Cómo se cumple
RNF-01	Reproducibilidad: mismos datos para todo el equipo	Scripts SQL fijos versionados en Git y entorno Docker; se verificó con `CHECKSUM_AGG` que dos instalaciones limpias dan datos idénticos
RNF-02	Integridad	Claves primarias, foráneas, `UNIQUE` en CUIT y DNI, y triggers de negocio
RNF-03	Rendimiento	7 índices sobre claves foráneas y fecha
RNF-04	Seguridad	3 roles; quien carga datos lo hace solo por procedimientos
RNF-05	Recuperabilidad	Backup completo con `CHECKSUM` y verificación con `RESTORE VERIFYONLY`
RNF-06	Mantenibilidad	Scripts idempotentes (`CREATE OR ALTER`) y organizados por carpeta

5. Diseño de la Base de Datos

5.1.Modelo Conceptual

El modelo tiene siete entidades y seis relaciones uno-a-muchos. No hay relaciones muchos-a-muchos puras: la relación entre clientes y calibraciones se resuelve con la entidad **Calificación**, que además aporta el atributo propio (puntaje) y la fecha.

- Un **Proveedor** provee muchos **Lotes**; cada lote lo provee un solo proveedor.
- Un **Grano** se compra en muchos **Lotes**; cada lote es de un único grano.
- Un **Lote** se usa en muchas **Calibraciones**.
- Un **Barista** realiza muchas **Calibraciones**.
- Una **Calibración** recibe muchas **Calificaciones**.
- Un **Cliente** deja muchas **Calificaciones**.

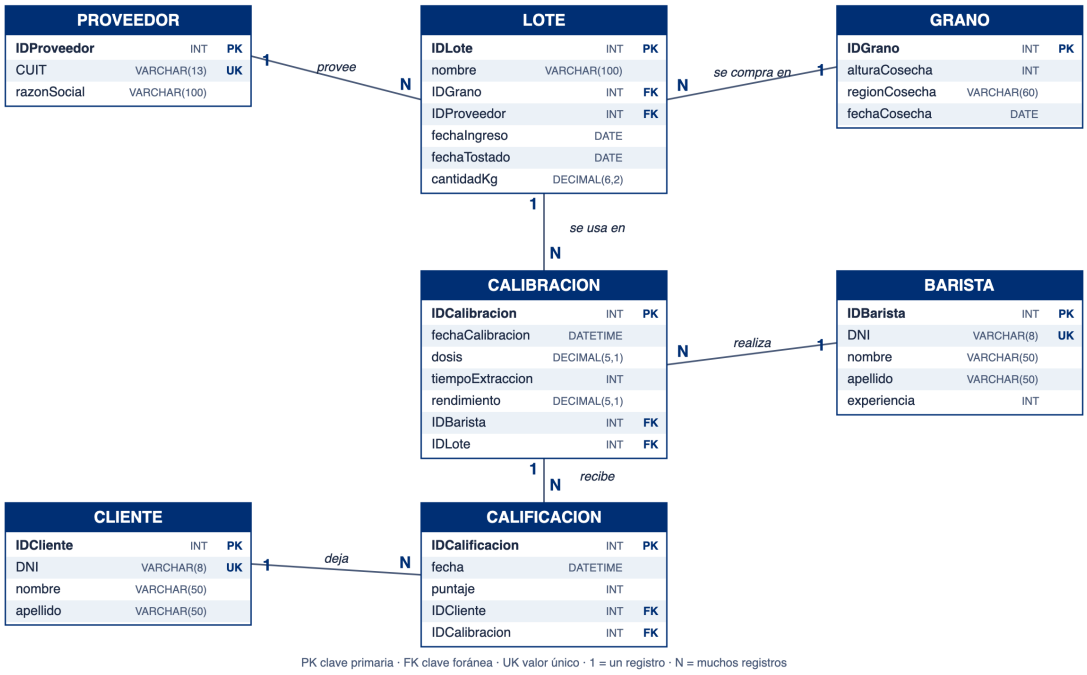


Figura 1. Diagrama entidad-relación de Cafetear

5.2.Modelo Lógico

Pasaje a tablas (clave primaria **PK**, foránea **FK**, única **UK**):

Tabla	Atributos
Proveedor	**IDProveedor** (PK), CUIT (UK), razonSocial
Grano	**IDGrano** (PK), alturaCosecha, regionCosecha, fechaCosecha
Lote	**IDLote** (PK), nombre, IDGrano (FK), IDProveedor (FK), fechaIngreso, fechaTostado, cantidadKg
Barista	**IDBarista** (PK), DNI (UK), nombre, apellido, experiencia
Cliente	**IDCliente** (PK), DNI (UK), nombre, apellido
Calibracion	**IDCalibracion** (PK), fechaCalibracion, dosis, tiempoExtraccion, rendimiento, IDBarista (FK), IDLote (FK)
Calificacion	**IDCalificacion** (PK), fecha, puntaje, IDCliente (FK), IDCalibracion (FK)

5.3.Modelo Físico

Script `01_crear_base_y_tablas.sql` (T-SQL, SQL Server 2022). Las tablas con claves que carga el usuario (Grano, Lote, Calibracion y Calificacion) usan `INT PRIMARY KEY` y el siguiente ID lo calculan los procedimientos; Proveedor, Barista y Cliente usan `IDENTITY`.

```
    razonSocial VARCHAR(100) NOT NULL
);

CREATE TABLE Grano (
    IDGrano       INT PRIMARY KEY,
    alturaCosecha INT          NOT NULL,   -- metros sobre el nivel del mar
    regionCosecha VARCHAR(60)  NOT NULL,
    fechaCosecha  DATE         NOT NULL
);

CREATE TABLE Lote (
    IDLote       INT PRIMARY KEY,
    nombre       VARCHAR(100) NOT NULL,   -- nombre de la finca, microlote o proceso
    IDGrano      INT NOT NULL REFERENCES Grano(IDGrano),
    IDProveedor  INT NOT NULL REFERENCES Proveedor(IDProveedor),
    fechaIngreso DATE NOT NULL,
    fechaTostado DATE NOT NULL,
    cantidadKg   DECIMAL(6,2) NOT NULL
);

CREATE TABLE Barista (
    IDBarista   INT IDENTITY(1,1) PRIMARY KEY,
    DNI         VARCHAR(8)  NOT NULL UNIQUE,
    nombre      VARCHAR(50) NOT NULL,
    apellido    VARCHAR(50) NOT NULL,
```

```
    experiencia INT         NOT NULL       -- años de experiencia
);

CREATE TABLE Cliente (
    IDCliente INT IDENTITY(1,1) PRIMARY KEY,
    DNI       VARCHAR(8)  NOT NULL UNIQUE,
    nombre    VARCHAR(50) NOT NULL,
    apellido  VARCHAR(50) NOT NULL
);

CREATE TABLE Calibracion (
    IDCalibracion    INT PRIMARY KEY,
    fechaCalibracion DATETIME     NOT NULL,
    dosis            DECIMAL(5,1) NOT NULL,   -- gramos de café molido
    tiempoExtraccion INT          NOT NULL,   -- segundos
    rendimiento      DECIMAL(5,1) NOT NULL,   -- gramos de café en la taza
    IDBarista        INT NOT NULL REFERENCES Barista(IDBarista),
    IDLote           INT NOT NULL REFERENCES Lote(IDLote)
);

CREATE TABLE Calificacion (
    IDCalificacion INT PRIMARY KEY,
    fecha          DATETIME NOT NULL,
    puntaje        INT      NOT NULL,
    IDCliente      INT NOT NULL REFERENCES Cliente(IDCliente),
    IDCalibracion  INT NOT NULL REFERENCES Calibracion(IDCalibracion)
);
GO
```

5.4.Diccionario de Datos

Tabla Proveedor

Campo	Tamaño (bytes)	Tipo	Descripción	NULL	Clave	Restricción
IDProveedor	4	INT	Identificador (IDENTITY)	No	PK	Autoincremental
CUIT	13	VARCHAR	CUIT del proveedor	No	UK	Único
razonSocial	100	VARCHAR	Nombre de la empresa	No		

Tabla Grano

Campo	Tamaño (bytes)	Tipo	Descripción	NULL	Clave	Restricción
IDGrano	4	INT	Identificador del grano	No	PK	
alturaCosecha	4	INT	Metros sobre el nivel del mar	No		
regionCosecha	60	VARCHAR	Región de origen	No		
fechaCosecha	3	DATE	Fecha de cosecha	No		

Tabla Lote

Campo	Tamaño (bytes)	Tipo	Descripción	NULL	Clave	Restricción
IDLote	4	INT	Identificador del lote	No	PK	
nombre	100	VARCHAR	Finca, microlote o proceso	No		
IDGrano	4	INT	Grano del lote	No	FK → Grano	
IDProveedor	4	INT	Quien lo vendió	No	FK → Proveedor	
fechaIngreso	3	DATE	Cuándo llegó al local	No		Trigger: no calibrar antes
fechaTostado	3	DATE	Fecha de tostado	No		
cantidadKg	5	DECIMAL(6, 2)	Kilos recibidos	No		Trigger: tope de uso

Tabla Barista

Campo	Tamaño (bytes)	Tipo	Descripción	NULL	Clave	Restricción
IDBarista	4	INT	Identificador (IDENTITY)	No	PK	Autoincremental
DNI	8	VARCHAR	Documento	No	UK	Único
nombre	50	VARCHAR	Nombre	No		
apellido	50	VARCHAR	Apellido	No		
experiencia	4	INT	Años de experiencia	No		≥ 0 (procedimiento)

Tabla Cliente

Campo	Tamaño (bytes)	Tipo	Descripción	NULL	Clave	Restricción
IDCliente	4	INT	Identificador (IDENTITY)	No	PK	Autoincremental
DNI	8	VARCHAR	Documento	No	UK	Único
nombre	50	VARCHAR	Nombre	No		
apellido	50	VARCHAR	Apellido	No		

Tabla Calibracion

Campo	Tamaño (bytes)	Tipo	Descripción	NULL	Clave	Restricción
IDCalibracion	4	INT	Identificador	No	PK	
fechaCalibracion	8	DATETIME	Momento de la prueba	No		
dosis	5	DECIMAL(5, 1)	Gramos de café molido	No		> 0 (procedimiento)
tiempoExtraccion	4	INT	Segundos de extracción	No		
rendimiento	5	DECIMAL(5, 1)	Gramos de café en la taza	No		
IDBarista	4	INT	Quién calibró	No	FK → Barista	
IDLote	4	INT	Lote usado	No	FK → Lote	

Tabla Calificacion

Campo	Tamaño (bytes)	Tipo	Descripción	NULL	Clave	Restricción
IDCalificacion	4	INT	Identificador	No	PK	
fecha	8	DATETIME	Cuándo calificó	No		Trigger: ≥ calibración
puntaje	4	INT	Estrellas de 1 a 5	No		Trigger: 1 a 5
IDCliente	4	INT	Quién calificó	No	FK → Cliente	Trigger: una por calibración
IDCalibracion	4	INT	Calibración calificada	No	FK → Calibracion	

5.5.Dependencias Funcionales

Todas las tablas están en **Tercera Forma Normal (3FN)**: cada atributo no clave depende de la clave completa y solo de ella. Las dependencias principales son:

- IDProveedor → CUIT, razonSocial (y CUIT → IDProveedor, por ser clave candidata).

- IDGrano → alturaCosecha, regionCosecha, fechaCosecha.
- IDLote → nombre, IDGrano, IDProveedor, fechaIngreso, fechaTostado, cantidadKg.
- IDBarista → DNI, nombre, apellido, experiencia. IDCliente → DNI, nombre, apellido.
- IDCalibracion → fechaCalibracion, dosis, tiempoExtraccion, rendimiento, IDBarista, IDLote.
- IDCalificacion → fecha, puntaje, IDCliente, IDCalibracion.

La región y la altura dependen del grano, no del lote; por eso viven en **Grano** y no se repiten en cada lote. La relación de extracción y el puntaje promedio son datos derivados y no se almacenan: se calculan con una función y vistas, para evitar redundancia.

6. Implementación y Scripts SQL

6.1. Inserción de Datos

Los datos los genera `herramientas/generar_datos.py` con una semilla fija y quedan escritos en scripts `.sql` (02, 03 y 04). Como los scripts son fijos y están en Git, todo el equipo carga exactamente los mismos datos. Las calificaciones no son aleatorias puras: la calidad sube con la curva de aprendizaje, cae en meses con eventos (cambio de molino, ola de calor) y hay lotes buenos, malos y en declive, para que las consultas tengan algo interesante que mostrar.

Filas cargadas (consulta real sobre la base):

tabla	filas
Proveedor	10
Grano	10
Lote	40
Barista	10
Cliente	60
Calibracion	225
Calificacion	346

Ejemplo de carga de maestros:

```
-- Cafetear · Datos maestros
-- Proveedor, Grano, Lote, Barista y Cliente. Ejecutar después de 01 y antes de 03.
```

```
USE Cafetear;
GO
SET NOCOUNT ON;


-- Proveedor (los CUIT son inventados)

INSERT INTO Proveedor (CUIT, razonSocial) VALUES
    (N'30-71458230-4', N'Café Andino S.R.L.'),
    (N'30-70912345-1', N'Tostadores del Sur S.A.'),
    (N'30-71203344-7', N'Finca Los Altos S.A.'),
```

6.2.Consultas SQL

Se usan solo contenidos vistos en la materia: `JOIN`, `GROUP BY`, `HAVING`, subconsultas, `EXISTS` y vistas. Los resultados que siguen se obtuvieron ejecutando las consultas sobre la base cargada.

1. Puntaje promedio por lote, con al menos 5 calificaciones (los 5 mejores)

```
SELECT TOP 5 l.nombre AS lote, COUNT(k.IDCalificacion) AS calificaciones,
       CAST(AVG(k.puntaje * 1.0) AS DECIMAL(3,2)) AS promedio
FROM Lote l
JOIN Calibracion c ON c.IDLote = l.IDLote
JOIN Calificacion k ON k.IDCalibracion = c.IDCalibracion
GROUP BY l.nombre
HAVING COUNT(k.IDCalificacion) >= 5
ORDER BY promedio DESC
```

lote	calificaciones	promedio
Gedeb Anaeróbico	7	5
Buesaco Lavado	9	4.56
Shantawene Natural	8	4.25
Villa Rica Natural	8	4.25
Cerro Azul Lavado	6	4.17

2. Evolución mensual de la calidad (mes/año y puntaje promedio)

```
SELECT CAST(MONTH(fecha) AS VARCHAR) + '/' + CAST(YEAR(fecha) AS VARCHAR) AS mes,
       COUNT(*) AS calificaciones,
       CAST(AVG(puntaje * 1.0) AS DECIMAL(3,2)) AS promedio
FROM Calificacion
WHERE fecha >= '2025-12-01' AND fecha < '2026-07-01'
```

```
GROUP BY YEAR(fecha), MONTH(fecha)
ORDER BY YEAR(fecha), MONTH(fecha)
```

mes	calificaciones	promedio
12/2025	18	1.44
1/2026	21	3.48
2/2026	14	3.93
3/2026	23	3.65
4/2026	27	3.85
5/2026	18	3.83
6/2026	13	2.69

Consultas avanzadas (subconsultas, `NOT EXISTS`, división relacional):

3. Receta de referencia de cada grano, sin la vista (subconsulta correlacionada)

```
SELECT g.IDGrano, c.IDCalibracion, c.dosis, c.tiempoExtraccion, c.rendimiento
FROM Grano g
JOIN Lote l ON l.IDGrano = g.IDGrano
JOIN Calibracion c ON c.IDLote = l.IDLote
WHERE c.IDCalibracion = (SELECT TOP 1 v.IDCalibracion
                         FROM vw_PuntajePorCalibracion v
                         WHERE v.IDGrano = g.IDGrano AND v.calificaciones >= 3
                         ORDER BY v.puntajePromedio DESC, v.IDCalibracion)
ORDER BY g.IDGrano
```

IDGrano	IDCalibracion	dosis	tiempoExtraccion	rendimiento
1	145	20.3	31	42.1
2	200	20.5	28	42.2
3	208	18.2	29	35
4	153	18.3	26	39.3
5	114	17	30	35.8
6	135	19.1	29	40.3
7	108	19.6	31	41.8

IDGrano	IDCalibracion	dosis	tiempoExtraccion	rendimiento
8	218	18.2	30	34.7

4. Lotes que nunca se calibraron (NOT EXISTS)

```
SELECT l.IDLote, l.nombre
FROM Lote l
WHERE NOT EXISTS (SELECT 1 FROM Calibracion c WHERE c.IDLote = l.IDLote)
```

IDLote	nombre
2	Finca Los Volcanes Lavado
6	Finca La Pastora Honey
7	Kochere Lavado
15	Aricha Natural
22	Santa María Lavado
27	Hambela Lavado
40	El Tablón Natural

5. Clientes que probaron el café de todos los baristas (división relacional)

```
SELECT cl.IDCliente, cl.nombre, cl.apellido
FROM Cliente cl
WHERE NOT EXISTS (SELECT 1
                  FROM Barista b
                  WHERE NOT EXISTS (SELECT 1
                                    FROM Calificacion k
                                    JOIN Calibracion c ON c.IDCalibracion =
k.IDCalibracion
                                    WHERE k.IDCliente = cl.IDCliente AND c.IDBarista
= b.IDBarista))
```

IDCliente	nombre	apellido
1	Daniela	Pérez
2	Carolina	Sánchez
3	Romina	Romero

IDCliente	nombre	apellido
5	Yamila	Gómez

6.3.Triggers y Vistas

Triggers. Aplican las reglas de negocio dentro de la base: valen aunque alguien inserte directamente, sin pasar por un procedimiento. Ante una violación hacen `ROLLBACK` y lanzan un error con `THROW`.

```
-- Control de calificaciones: puntaje de 1 a 5, no anterior a la calibración y una
sola por cliente y calibración.
CREATE OR ALTER TRIGGER dbo.trg_Calificacion_Control
ON dbo.Calificacion
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (SELECT 1 FROM inserted WHERE puntaje NOT BETWEEN 1 AND 5)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50020, N'El puntaje debe estar entre 1 y 5.', 1;
    END;

    IF EXISTS (SELECT 1
               FROM inserted i
               JOIN Calibracion c ON c.IDCalibracion = i.IDCalibracion
               WHERE i.fecha < c.fechaCalibracion)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50021, N'La calificación no puede ser anterior a la calibración.', 1;
    END;

    IF EXISTS (SELECT 1
               FROM Calificacion k
               JOIN inserted i ON i.IDCliente = k.IDCliente AND i.IDCalibracion =
k.IDCalibracion
               GROUP BY k.IDCliente, k.IDCalibracion
               HAVING COUNT(*) > 1)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50022, N'Ese cliente ya calificó esta calibración.', 1;
    END;
END
GO
```

Vistas. Se crearon cuatro. `vw_PuntajePorCalibracion` es la base de las demás; `vw_MejorCalibracionPorGrano` usa `NOT EXISTS` para quedarse con la mejor calibración de cada grano (con al menos 3 calificaciones, para que el promedio sea confiable).

```
CREATE OR ALTER VIEW dbo.vw_MejorCalibracionPorGrano AS
SELECT g.IDGrano, g.regionCosecha, v.IDCalibracion, v.fechaCalibracion, v.dosis,
v.tiempoExtraccion,
       v.rendimiento, v.relacionExtraccion, v.calificaciones, v.puntajePromedio
FROM vw_PuntajePorCalibracion v
JOIN Grano g ON g.IDGrano = v.IDGrano
WHERE v.calificaciones >= 3
  AND NOT EXISTS (SELECT 1
                  FROM vw_PuntajePorCalibracion otra
                  WHERE otra.IDGrano = v.IDGrano
                    AND otra.calificaciones >= 3
                    AND otra.puntajePromedio > v.puntajePromedio);
GO
```

Resultado de la vista `vw_MejorCalibracionPorGrano`:

IDGrano	regionCosecha	IDCalibracion	dosis	tiempoExtraccion	rendimiento	relacion	calif	puntaje
1	Huila, Colombia	145	20.3	31	42.1	2.07	4	5
2	Cerrado Mineiro, Brasil	200	20.5	28	42.2	2.06	4	4.5
3	Yirgacheffe, Etiopía	220	18.8	30	35.7	1.9	3	5
3	Yirgacheffe, Etiopía	208	18.2	29	35	1.92	4	5
4	Antigua, Guatemala	153	18.3	26	39.3	2.15	7	4.57
5	Sul de Minas, Brasil	114	17	30	35.8	2.11	6	4.33
6	Nariño, Colombia	135	19.1	29	40.3	2.11	6	4.83
7	Sidamo, Etiopía	108	19.6	31	41.8	2.13	4	4.25
7	Sidamo, Etiopía	138	17.1	28	34	1.99	4	4.25
8	Tarrazú, Costa Rica	218	18.2	30	34.7	1.91	8	4.13

6.4.Funciones y Procedimientos

Funciones (`01_funciones.sql`): `fn_RelacionExtraccion` (escalar), `fn_PuntajePromedioLote` (escalar), `fn_DiasDesdeTostado` (escalar) y `fn_CalibracionesDeBarista` (tabla en línea).

```
-- Cafetear · Funciones

USE Cafetear;
GO

-- Relación de extracción: gramos de café en la taza por cada gramo de café molido.
Con 18 g y 36 g da 2,00 (1:2).
CREATE OR ALTER FUNCTION dbo.fn_RelacionExtraccion (@dosis DECIMAL(5,1),
@rendimiento DECIMAL(5,1))
RETURNS DECIMAL(5,2)
AS
BEGIN
    RETURN CASE WHEN @dosis > 0 THEN CAST(@rendimiento / @dosis AS DECIMAL(5,2))
END;
END
GO

-- Puntaje promedio que recibió un lote, sumando las calificaciones de todas sus
calibraciones.
CREATE OR ALTER FUNCTION dbo.fn_PuntajePromedioLote (@IDLote INT)
RETURNS DECIMAL(4,2)
AS
BEGIN
    RETURN (SELECT CAST(AVG(k.puntaje * 1.0) AS DECIMAL(4,2))
            FROM Calibracion c
            JOIN Calificacion k ON k.IDCalibracion = c.IDCalibracion
            WHERE c.IDLote = @IDLote);
END
GO
```

Procedimientos. Cada tabla tiene sus cinco procedimientos (`Crear`, `Obtener`, `Listar`, `Actualizar`, `Eliminar`), escritos de forma simple: validan lo necesario con `THROW`, calculan el siguiente ID cuando la tabla no es `IDENTITY` y `Crear` devuelve el ID generado.

```
-- Cafetear · CRUD de Calificacion (procedimientos almacenados)

USE Cafetear;
GO

-- Crear
CREATE OR ALTER PROCEDURE usp_Calificacion_Crear
    @fecha DATETIME,
    @puntaje INT,
    @IDCliente INT,
    @IDCalibracion INT
AS
BEGIN
    IF @puntaje NOT BETWEEN 1 AND 5
        THROW 50000, N'El puntaje debe estar entre 1 y 5.', 1;
    IF EXISTS (SELECT 1 FROM Calificacion WHERE IDCliente = @IDCliente AND
IDCalibracion = @IDCalibracion)
        THROW 50000, N'Ese cliente ya calificó esta calibración.', 1;
```

```
    DECLARE @IDCalificacion INT = (SELECT ISNULL(MAX(IDCalificacion), 0) + 1 FROM
Calificacion);

    INSERT INTO Calificacion (IDCalificacion, fecha, puntaje, IDCliente,
IDCalibracion)
    VALUES (@IDCalificacion, @fecha, @puntaje, @IDCliente, @IDCalibracion);

    SELECT @IDCalificacion AS IDCalificacion;
```

Además hay dos procedimientos de reporte, `usp_RankingBaristas` (con período opcional) y `usp_ReporteLote`:

```
EXEC usp_RankingBaristas @desde = '2026-01-01', @hasta =
'2026-10-31';
```

IDBarista	barista	calibraciones	calificaciones	puntajePromedio
6	Julián Acosta	5	6	4.33
2	Lucas Benítez	22	41	4.24
5	Valentina Pereyra	14	28	4.04
1	Martina Ríos	33	68	3.9
7	Camila Ferrer	9	20	3.65
9	Florencia Montero	9	11	3.45
10	Tomás Arias	8	14	3.43
3	Sofía Carrizo	18	35	2.97
4	Mateo Duarte	9	14	2.36

7. Interfaz de Usuario

Este trabajo no incluye interfaz gráfica: el alcance es la base de datos. La interacción prevista es mediante procedimientos, de modo que una futura aplicación (por ejemplo una web en FastAPI) solo tenga que llamar a `EXEC`. Ejemplo del flujo de un barista:

```
EXEC usp_Lote_Listar;                                -- elige el lote que va a
calibrar
EXEC usp_Calibracion_Crear @fechaCalibracion = '2026-11-05T08:30:00', @dosis = 18.0,
                           @tiempoExtraccion = 28, @rendimiento = 36.0, @IDBarista =
1, @IDLote = 3;
SELECT * FROM vw_CalibracionesDeHoy;                -- ve lo que se calibró hoy
```

8. Plan de Desarrollo y Metodología

Se trabajó con **SCRUM** en tres sprints cortos. El código y los scripts se versionaron en un repositorio privado de GitHub (`joacomorales25/cafetear`), con ramas y revisión por Pull Request.

Sprint	Objetivo	Entregables
1	Entender el dominio y diseñar	Propuesta, DER, modelo lógico, diccionario de datos
2	Construir	DDL, datos de prueba, procedimientos CRUD, entorno Docker
3	Cerrar y validar	Vistas, funciones, triggers, índices, seguridad, backup, pruebas e informe

Roles: Product Owner (definición del negocio y prioridades), Scrum Master (coordinación) y equipo de desarrollo (los seis integrantes, que escriben y revisan consultas). Herramientas: SQL Server 2022 en Docker, Git/GitHub, Python para generar datos.

9. Optimización y Rendimiento

SQL Server no crea índices automáticamente sobre las claves foráneas. Se crearon siete, sobre las columnas que más se usan en `JOIN` y filtros (`05_indices.sql`):

indice	tabla	columna
IX_Calibracion_Fecha	Calibracion	fechaCalibracion
IX_Calibracion_IDBarista	Calibracion	IDBarista
IX_Calibracion_IDLote	Calibracion	IDLote
IX_Calificacion_IDCalibracion	Calificacion	IDCalibracion
IX_Calificacion_IDCliente	Calificacion	IDCliente
IX_Lote_IDGrano	Lote	IDGrano
IX_Lote_IDProveedor	Lote	IDProveedor

Medición. Se comparó una consulta de promedio por lote con y sin los índices, usando `SET STATISTICS IO ON`: con índices leyó 1.887 páginas lógicas y sin ellos 1.953. La mejora es pequeña, y es lo esperable: con 225 calibraciones y 346 calificaciones las tablas caben en pocas páginas y el optimizador casi no necesita índices. Su beneficio real aparece cuando el historial crece (más sucursales y años de calibraciones); se dejaron creados porque el costo es mínimo y evitan degradación futura.

10.Seguridad y Recuperación

10.1.Roles y permisos

Rol	Puede	No puede
rol_barista	Ejecutar los procedimientos de calibración y de consulta de lotes y granos; leer dos vistas	Leer ni modificar tablas directamente; borrar clientes
rol_analista	Leer todo (tablas y vistas) y ejecutar los dos reportes	Insertar, actualizar o borrar (`DENY`)
rol_administrador	Leer todo y ejecutar todos los procedimientos	—

El barista no tiene permiso sobre las tablas: toda carga pasa por procedimientos (encadenamiento de propiedad). Prueba real con un usuario de prueba en `rol_barista`:

```
OK: el barista puede ejecutar usp_Lote_Obtener
OK: The SELECT permission was denied on the object 'Calificacion', database
'Cafetear', schema 'dbo'.
OK: The EXECUTE permission was denied on the object 'usp_Cliente_Eliminar', database
'Cafetear', schema 'dbo'.
```

10.2.Respaldo y recuperación

Se genera una copia completa con suma de verificación y luego se valida que se pueda restaurar (`mantenimiento/backup_y_restauracion.sql`):

```
-- Cafetear · Copia de seguridad y restauración
-- Se ejecuta a mano (no forma parte de la carga automática).
-- La ruta es la de dentro del contenedor. Para sacar el archivo: docker cp
cafetear_sqlserver:/var/opt/mssql/data/Cafetear.bak .

-- 1) Copia completa, con control de integridad.
BACKUP DATABASE Cafetear
TO DISK = N'/var/opt/mssql/data/Cafetear.bak'
WITH INIT, CHECKSUM, STATS = 25;
GO
```

```
-- 2) Verificar que la copia se puede restaurar (no toca la base).
RESTORE VERIFYONLY
```

Resultado medido: 762 páginas respaldadas en 0,155 segundos y `RESTORE VERIFYONLY` informó que el respaldo es válido. La restauración completa queda documentada, comentada, en el mismo script, porque pisa la base.

11.Pruebas y Validación

Hay dos scripts de prueba que dejan la base como estaba: `probar_crud.sql` (alta, consulta, modificación y baja de cada tabla, más errores esperados) y `probar_objetos.sql` (funciones, vistas, reportes, triggers y permisos). Cada línea debe decir `OK`. Resultados reales:

Caso	Resultado obtenido
Proveedor con CUIT repetido	Violation of UNIQUE KEY constraint … duplicate key value is (30-99999999-9)
Lote con un grano inexistente	The INSERT statement conflicted with the FOREIGN KEY constraint … table "dbo.Grano"
Lote con cantidad en kilos = 0	La cantidad en kilos debe ser mayor a cero.
Barista con experiencia negativa	La experiencia no puede ser negativa.
Calibración con dosis = 0	La dosis debe ser mayor a cero.
Calificación con puntaje 9 (trigger)	El puntaje debe estar entre 1 y 5.
Calificación anterior a la calibración (trigger)	La calificación no puede ser anterior a la calibración.
Mismo cliente califica dos veces la misma calibración (trigger)	Ese cliente ya calificó esta calibración.
Calibrar antes del ingreso del lote (trigger)	No se puede calibrar con un lote antes de su ingreso.
Calibrar con 9 kg de café en un lote que no los tiene (trigger)	El lote no tiene café suficiente para esta calibración.
Borrar un proveedor con lotes	The DELETE statement conflicted with the REFERENCE constraint … table "dbo.Lote"
Borrar una calibración ya calificada	The DELETE statement conflicted with the REFERENCE constraint … table "dbo.Calificacion"
Estado final de la base	OK: la base quedó igual que antes (701 filas en total).

12.Gestión de Riesgos

Riesgo	Probabilidad	Impacto	Mitigación
Que cada integrante tenga datos distintos	Alta	Medio	Scripts fijos en Git y Docker; verificación por checksum
Datos cargados con errores	Media	Alto	Triggers, claves y procedimientos con validación
Pérdida de datos	Baja	Alto	Backup con checksum y `RESTORE VERIFYONLY`
Acceso indebido a las tablas	Media	Alto	Roles y permisos solo por procedimientos
Crecimiento del volumen de datos	Media	Medio	Índices sobre claves foráneas y fecha
Cambios de alcance (por ejemplo mezclas de granos)	Media	Medio	Se dejó fuera de alcance; el modelo admite agregar una tabla Receta

13.Costos y Presupuesto

Estimación académica, calculada con horas del equipo (valores de referencia a confirmar por el grupo):

Concepto	Detalle	Costo estimado (USD)
Diseño y modelado	30 h × 6 integrantes a USD 8/h	1.440
Desarrollo y pruebas	40 h × 6 integrantes a USD 8/h	1.920
Documentación e informe	10 h × 6 integrantes a USD 8/h	480
Licencias	SQL Server Developer, Docker Desktop (uso personal), Git/GitHub (gratis)	0
Total		**3.840**

14.Conclusiones y Recomendaciones

Cafetear convierte la calibración, que hoy es intuición, en información consultable: permite saber qué receta funciona mejor con cada grano, ver si la calidad sube o baja mes a mes y detectar lotes o prácticas que dañan la experiencia del cliente. El modelo es simple a

propósito (siete tablas, sin relaciones muchos-a-muchos), y las reglas importantes viven en la base, no en la buena voluntad de quien carga los datos.

Recomendaciones y próximos pasos:

- Agregar una interfaz (web o app) que llame a los procedimientos desde el puesto del barista.
- Incorporar una tabla de sucursales para comparar locales entre sí.
- Evaluar mezclas de granos (tabla Receta) si el negocio las vuelve habituales.
- Registrar variables que hoy no se miden (temperatura, humedad, molienda) para explicar mejor las diferencias de calidad.
- Incorporar funciones de ventana (`ROW_NUMBER`, `LAG`) cuando se vean en la materia, para tendencias más finas.

Limitaciones: los datos son de prueba generados para simular un año de operación; las conclusiones numéricas ilustran el funcionamiento del sistema y no representan una cafetería real.

15.Referencias y Anexos

- Microsoft Learn: T-SQL (https://learn.microsoft.com/sql/t-sql) – `CREATE TRIGGER`, `CREATE VIEW`, `CREATE PROCEDURE`, `THROW`, `BACKUP DATABASE`.
- Material de la cátedra: clases 6 y 7 (vistas, subconsultas, triggers, procedimientos y funciones, índices) – Ing. Franco Emanuel Salazar.
- Repositorio del proyecto: https://github.com/joacomorales25/cafetear (scripts, datos, procedimientos, Docker y herramientas).
- Anexo: estructura del repositorio.

```
cafetear/
├── docker-compose.yml      entorno SQL Server + inicialización
├── herramientas/           generadores de datos y de procedimientos (Python)
├── queries/
│   ├── 01_…07_*.sql        tablas y datos
│   ├── crud/               35 procedimientos
│   ├── objetos/            funciones, vistas, triggers, índices, roles
│   ├── pruebas/            scripts de prueba
│   └── mantenimiento/      backup y restauración
└── informe/                este informe y el DER
```